



SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL
AND TECHNICAL SCIENCES



Metroid Maze Explorer: A Java-Based 2D Exploration Game Using Collections, Swing GUI, and JDBC

CSA0926 – PROGRAMMING IN JAVA

Review-1 01/08/2026

Guided by:

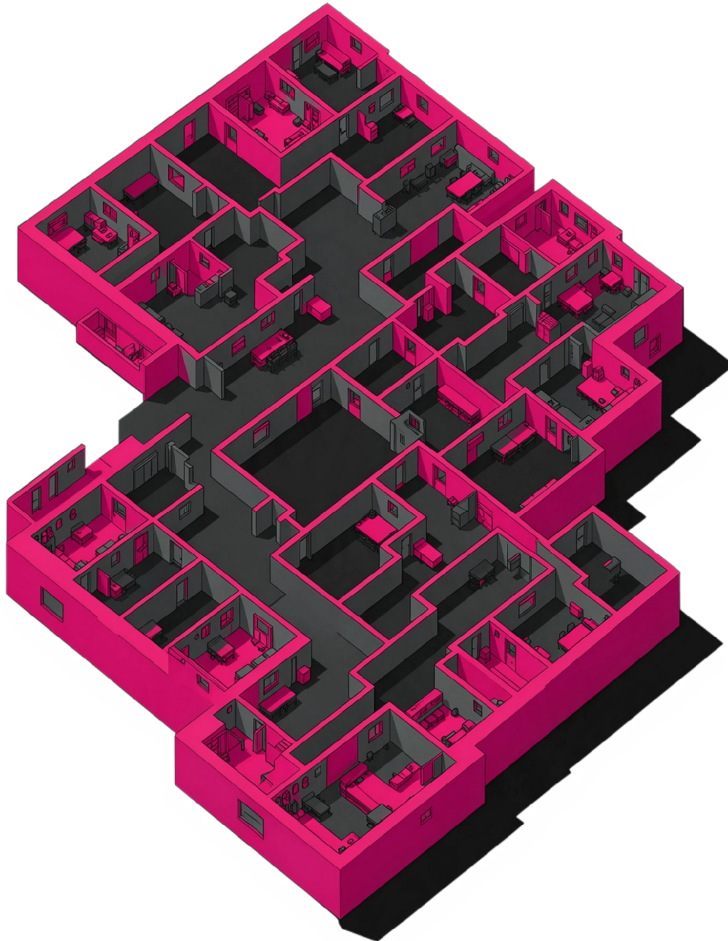
DR. M. RAMKUMAR

Presented by:

B. RAMYA [192424183]

U. JAYASUDHA [192411228]

Game Concept Overview



Core Game Loop

Navigate a maze, solve puzzles, and defeat enemies across interconnected rooms

Player Role

Explore rooms, collect power-ups, and find the exit before conditions are met

Win / Lose Condition

Reach the final chamber before time runs out or health fully depletes

Reference Game Study



Inspired by the classic **Metroid series** — action-adventure exploration built around discovery and backtracking.

- Key mechanic: Backtracking through unlocked areas with newly acquired abilities
- Simplified scope: No physics engine, basic collision detection only
- Focus: Understanding why original mechanics work — and which corners are being deliberately cut



Learning Objectives



2D Array Manipulation

Maze grid representation and spatial logic

Stack-Based Undo/Redo

Full action history for player move management

Multithreading

Animation and game loop running concurrently

JDBC Persistence

Game state and high score storage in MySQL

Exception Handling

Robust error recovery across all failure scenarios



PROPOSED MODULE BREAKDOWN



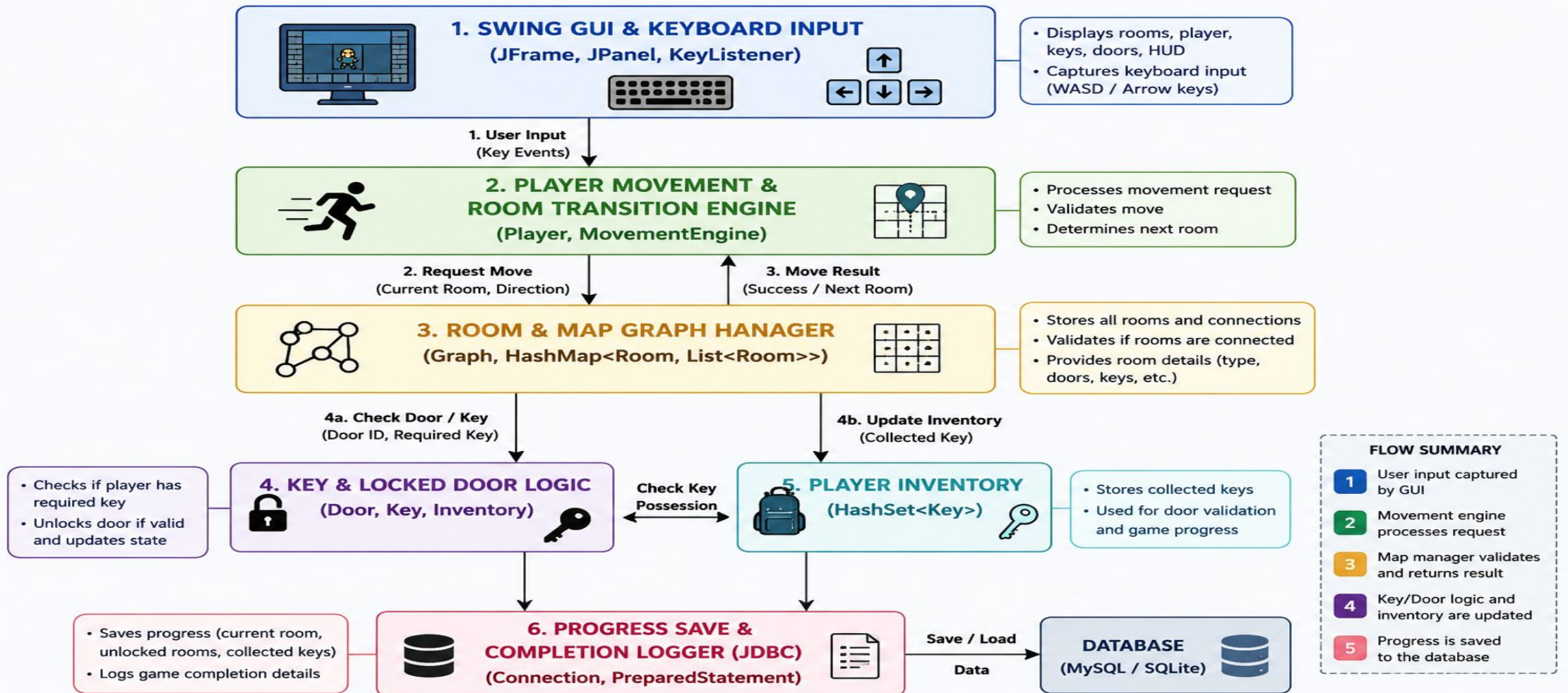
Module	What It Does	Java Feature / Core Class
Room & Map Graph Manager	Creates and manages interconnected rooms as a game map, tracking accessible and locked rooms.	Collections Framework – HashMap<Room, List<Room>>, ArrayList, Room class
Player Movement & Room Transition Engine	Handles player movement and transitions between connected rooms based on direction.	OOP & State Management – Player, MovementEngine, Room classes
Key & Locked Door Logic	Manages key collection, inventory tracking, and unlocking doors with the correct key.	Collections & OOP – HashSet<Key>, Door, Key, Inventory classes
Swing GUI & Keyboard Controls	Displays rooms, player, keys, and doors while processing keyboard inputs for movement.	Swing & Event Handling – JFrame, JPanel, KeyListener
Progress Save & Completion Logger	Saves game progress, collected keys, unlocked rooms, and completion records in a database.	JDBC – Connection, PreparedStatement, ResultSet



Engineer to Excel



System Architecture & Module Interaction





Tech Stack & Implementation



GUI Java Swing / AWT – rendering and event handling	Collections Java Collections Framework – inventory and entity lists	OOP Encapsulation and inheritance for all game entities
Exceptions Try-catch blocks for invalid moves and DB errors		Persistence JDBC + MySQL for persistent game state storage



PROPOSED DATABASE SCHEMA



Table: player_progress

Column Name	Data Type	Purpose
progress_id	INT (PK)	Unique progress record
player_name	VARCHAR(50)	Stores player name
current_room	VARCHAR(50)	Current room location
collected_keys	VARCHAR(100)	Keys collected by player
unlocked_rooms	VARCHAR(200)	Rooms unlocked so far
save_time	TIMESTAMP	Last save time

Table: completion_log

Column Name	Data Type	Purpose
completion_id	INT (PK)	Unique completion record
player_name	VARCHAR(50)	Stores player name
completion_time	VARCHAR(20)	Time taken to finish game
total_keys_collected	INT	Number of keys collected
completion_date	DATE	Date of completion

Data Storage Flow

Gameplay → Progress Saved → Goal Room Reached → Completion Logged

Multithreading Plan



Game Loop Thread

Core game logic and collision detection – the heartbeat of the engine



Animation Thread

Sprite updates and visual effects running independently of logic



AI Opponent Thread

Enemy behaviour and pathfinding without blocking the main loop



Countdown Thread

Timer and time-based events managed separately for precision



Justification: The GUI would **freeze** without separating these tasks into independent threads.

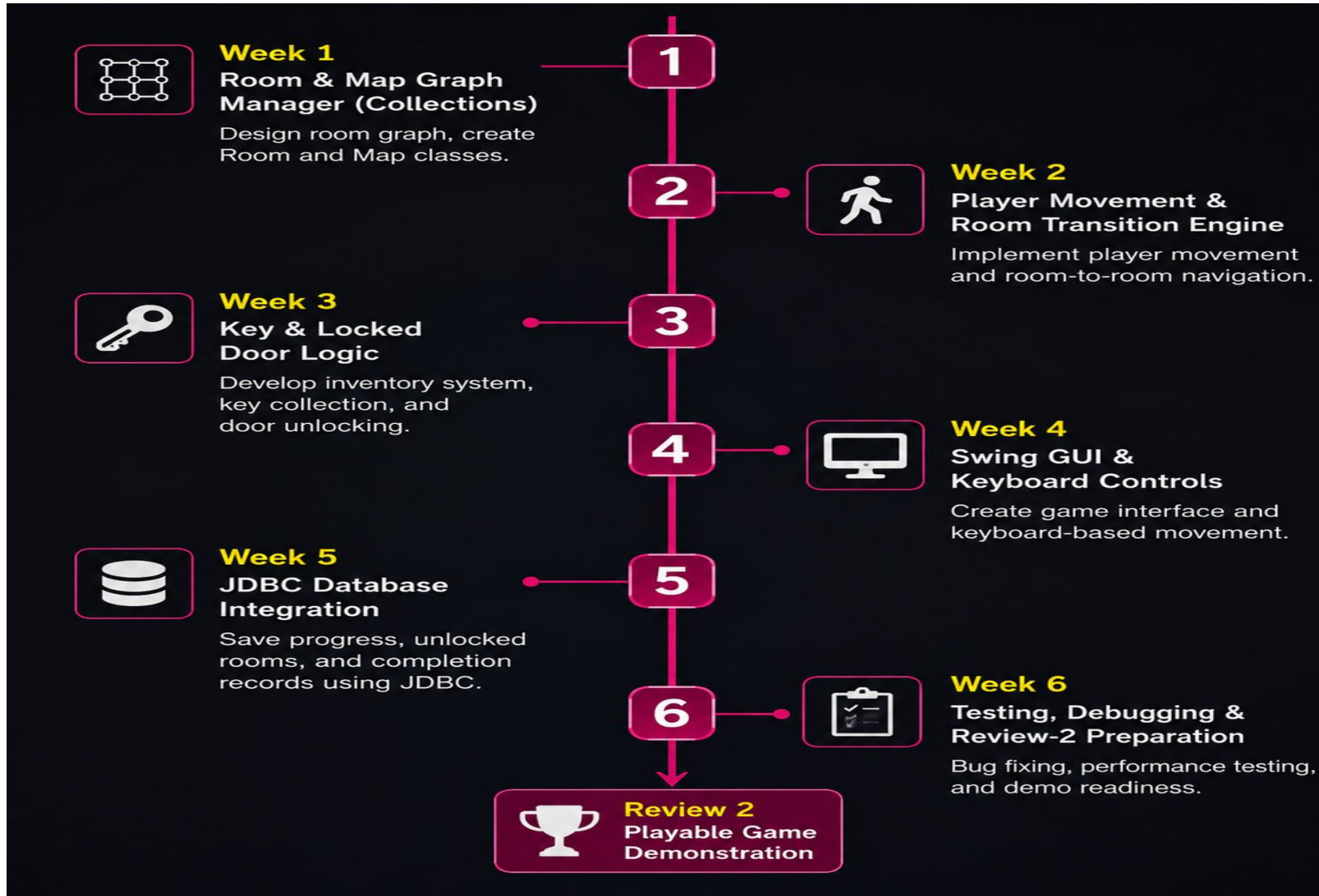


EXCEPTION HANDLING PLAN



Failure Scenario	Exception / Validation	Handling Strategy
Player tries to enter a locked room without the required key	Validation Check (if-else)	Show "Key Required" message and block room access.
Invalid room connection or missing room data	NullPointerException	Keep player in current room and display an error message.
Database connection fails while saving progress	SQLException	Show "Unable to Save Progress" and continue gameplay.
Unsupported keyboard input	Input Validation	Ignore invalid keys and accept only movement controls.
No saved game data found during load	SQLException / NullPointerException	Load default starting room and notify the player.

Work Plan & Timeline to Review 2



ANTICIPATED CHALLENGES

 TECHNICAL CHALLENGE	 POTENTIAL ISSUE	 PLANNED APPROACH
 Managing Room Graph Connections	Incorrect or missing room links may prevent proper navigation between rooms.	➤ Use graph structure (HashMap + ArrayList) to manage rooms. ➤ Thoroughly test all room connections.
 Key & Door State Synchronization	Doors may not unlock correctly after collecting keys.	➤ Maintain a centralized inventory system for keys. ➤ Validate key-door mapping before allowing room transitions.
 Database Save/Load Consistency	Progress data may not be stored or restored correctly, causing loss of game state.	➤ Use JDBC transactions for reliable operations. ➤ Implement exception handling and extensive save/load testing.



RISK MITIGATION / OUR APPROACH



Modular design
for easier debugging.



Incremental testing
after each module implementation.



Validation checks
before room transitions and database operations.

THANK YOU!